

CS224N Lecture 9: Pre-training (Part 1)

Stanford University

Week 5

Abstract

This lecture introduces the concept of pre-training in natural language processing, a paradigm shift that has revolutionized modern NLP. We begin with subword tokenization methods that address the limitations of fixed vocabularies, then explore how entire models (not just word embeddings) can be pre-trained on large amounts of unlabeled text. The lecture covers masked language modeling, BERT architecture, and the pre-training/fine-tuning paradigm that enables transfer learning across NLP tasks.

Contents

1	Motivation: From Word Embeddings to Model Pre-training	2
1.1	Review: Distributional Hypothesis	2
1.2	The Need for Contextual Representations	2
2	Subword Tokenization	2
2.1	The Problem with Fixed Vocabularies	2
2.2	Morphologically Rich Languages	3
2.3	Solution: Byte Pair Encoding (BPE)	3
2.3.1	BPE Algorithm	4
2.3.2	Example: Tokenizing with Subwords	4
2.3.3	Processing Subword Sequences	4
3	What Can We Learn from Predicting Masked Words?	5
3.1	Core Idea: Reconstructing Corrupted Input	5
3.2	Examples of Learnable Phenomena	5
4	Pre-training and Fine-tuning Paradigm	7
4.1	Traditional Approach: Pre-trained Embeddings Only	7
4.2	Modern Approach: Full Model Pre-training	7
4.3	Two-Stage Training Process	8
4.4	Mathematical Formulation	8
4.5	Why Does This Work?	9
4.6	Comparison with Training from Scratch	9
5	Language Modeling as Pre-training	9
5.1	Simple Approach: Standard Language Modeling	9
5.2	After Pre-training: Using the Model	10

6	Three Types of Model Architectures	10
6.1	Architecture Overview	10
6.2	Encoder Architecture	10
6.3	Decoder Architecture	11
6.4	Encoder-Decoder Architecture	11
7	Encoder Pre-training: Masked Language Modeling	12
7.1	Why Not Standard LM for Encoders?	12
7.2	Solution: Masked Language Modeling (MLM)	12
8	BERT: Bidirectional Encoder Representations from Transformers	13
8.1	Overview	13
8.2	BERT Architecture	14
8.3	Three Types of Embeddings	14
8.4	Masking Strategy	14
8.5	Next Sentence Prediction (NSP)	15
8.6	BERT Training Details	16
8.7	Fine-tuning BERT	16
9	BERT Evaluation: GLUE Benchmark	16
9.1	The GLUE Benchmark	16
9.2	BERT’s Impact	17
10	Limitations and Considerations	17
10.1	When to Use BERT	17
10.2	The [MASK] Token Problem	18
10.3	Computational Costs	18
11	Summary and Key Takeaways	18
11.1	Main Concepts	18
11.2	The Pre-training Revolution	19
11.3	Looking Ahead	19
12	Further Reading	19

1 Motivation: From Word Embeddings to Model Pre-training

1.1 Review: Distributional Hypothesis

Recall the foundational principle from earlier lectures:

“You shall know a word by the company it keeps.” — J.R. Firth

This distributional hypothesis underlies Word2Vec and other static embedding methods. However, these methods have a critical limitation: **one vector per word type**, regardless of context.

Example 1.1 (Polysemy Problem). The word “record” has multiple meanings:

- **Noun:** “I listened to the *record*.”
- **Verb:** “I will *record* the meeting.”

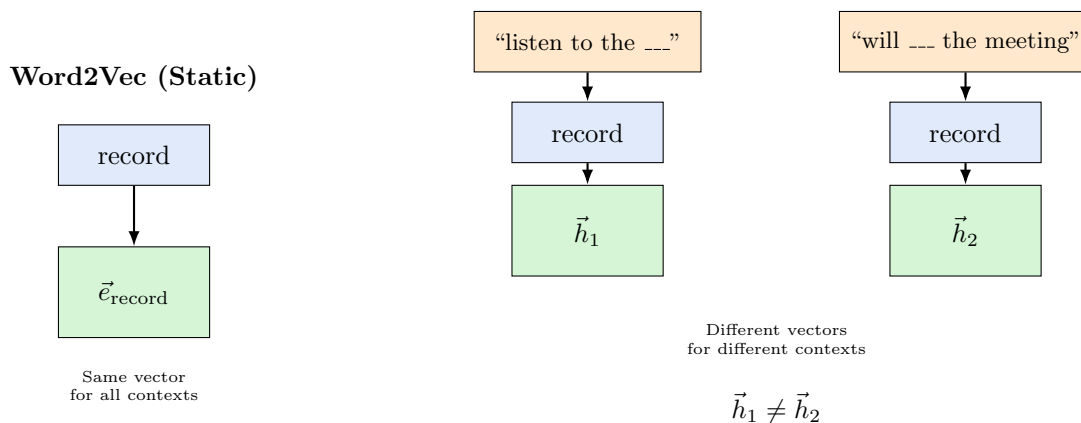
Word2Vec produces a single embedding that must somehow capture both meanings, resulting in a mixed representation.

1.2 The Need for Contextual Representations

“The complete meaning of a word is always contextual, and no study of meaning apart from a complete context can be taken seriously.” — J.R. Firth

Key Insight: We need representations that vary based on the specific context in which a word appears.

Contextual Representations



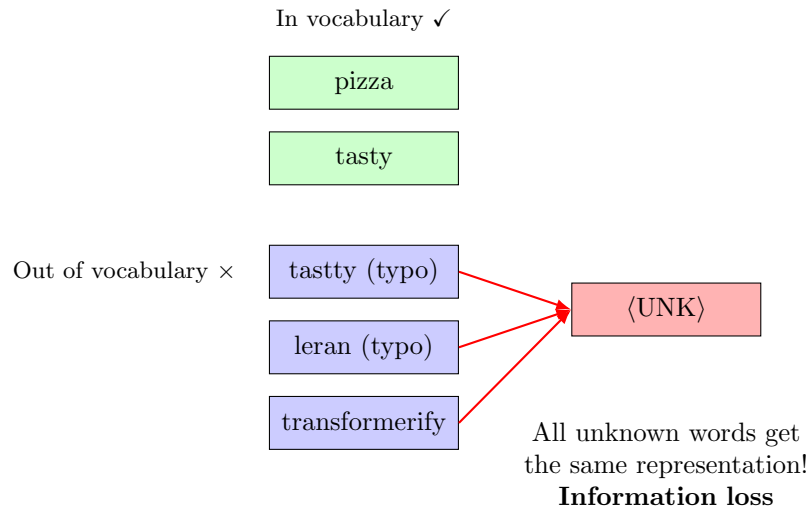
2 Subword Tokenization

2.1 The Problem with Fixed Vocabularies

Problem 2.1 (Out-of-Vocabulary Words). Traditional word-based vocabularies face several challenges:

- **Novel words:** New words constantly enter language (e.g., “transformerify”)
- **Typos:** Misspellings like “leran” instead of “learn”
- **Rare words:** Domain-specific or infrequent terms
- **Morphological variants:** Different inflections of the same root

All unknown words get mapped to a single $\langle \text{UNK} \rangle$ token, losing crucial information.



2.2 Morphologically Rich Languages

The problem is even more severe in morphologically rich languages:

Example 2.1 (Swahili Verb Conjugations). A single Swahili verb can have over **300 conjugations**. If we treat each as a separate vocabulary entry:

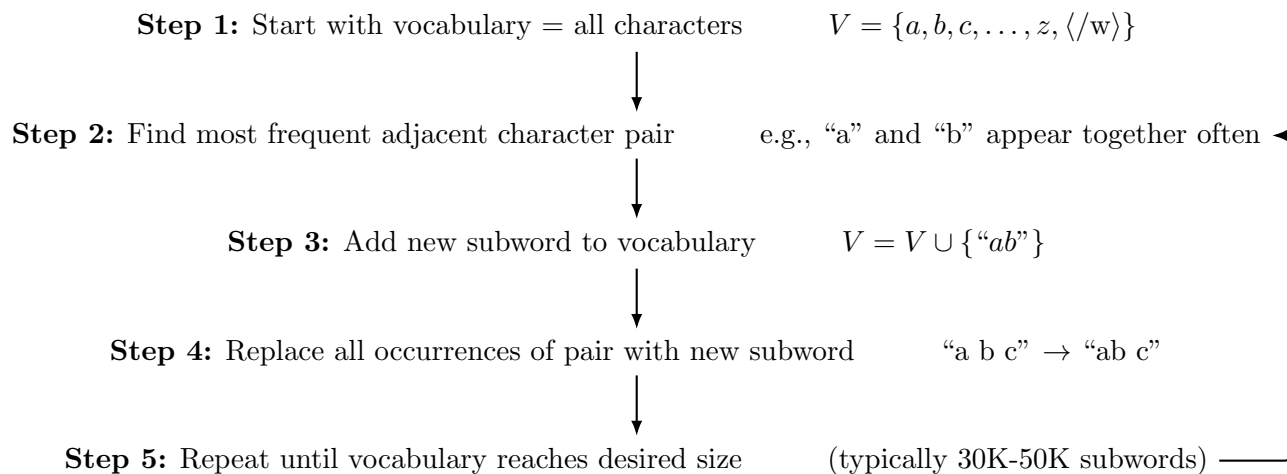
- Vocabulary explodes in size
- Each conjugation gets an independent vector
- No shared information about the common root
- Inefficient and poor generalization

2.3 Solution: Byte Pair Encoding (BPE)

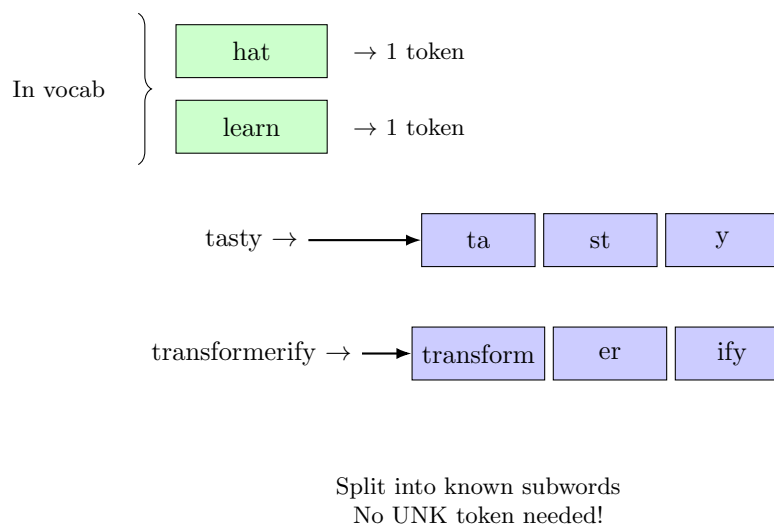
Definition 2.1 (Subword Tokenization). Instead of treating words as atomic units, split them into frequently occurring **subword units**. This allows:

- Representing any word as a sequence of known subwords
- Sharing information across morphological variants
- Handling novel words compositionally

2.3.1 BPE Algorithm



2.3.2 Example: Tokenizing with Subwords

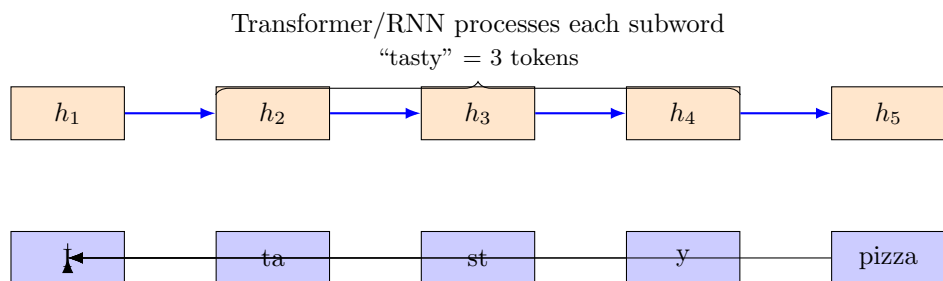


Key Properties:

- Common words appear as single tokens (efficient)
- Rare words split into multiple subwords (compositional)
- Novel words can always be represented (no UNK)
- Subword embeddings are learned and shared across words

2.3.3 Processing Subword Sequences

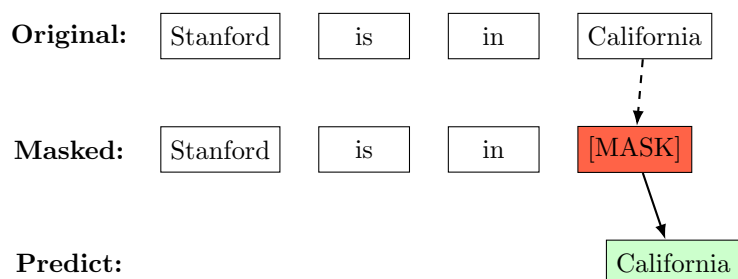
Once tokenized, subwords are processed by RNNs or Transformers as independent tokens:



3 What Can We Learn from Predicting Masked Words?

3.1 Core Idea: Reconstructing Corrupted Input

Hypothesis: By masking parts of text and training a model to reconstruct it, the model must learn rich representations of language and world knowledge.



This is a **self-supervised** learning task: the text itself provides the supervision signal.

3.2 Examples of Learnable Phenomena

Example 3.1 (Geographic Knowledge). **Masked:** “Stanford University is located in _____, California.”

What’s learned: The model must know:

- Stanford is a university
- Universities are located in cities/towns
- The word should be a place name
- Specific geographic fact: Stanford is in Palo Alto

Example 3.2 (Syntax). **Masked:** “I put _____ fork down on the table.”

Possible answers: the, a, my, this

What’s learned:

- Syntactic constraint: determiner before noun
- Agreement rules (singular “fork” requires singular determiner)
- Part-of-speech patterns

Example 3.3 (Coreference). **Masked:** “The woman walked across the street, checking for traffic over _____ shoulder.”

Answer: her

What’s learned:

- Coreference: “her” refers to “the woman”
- Possessive pronoun agreement
- Real-world knowledge: people check over their own shoulder

Example 3.4 (Semantic Categories). **Masked:** “I went to the ocean to see the fish, turtles, seals, and _____.”

Possible answers: whales, dolphins, sharks

What’s learned:

- Semantic categorization (marine animals)
- List structure and coordination
- World knowledge about ocean life

Example 3.5 (Sentiment). **Masked:** “Overall, the value I got from the two hours watching it was the sum total of the popcorn and drink. The movie was _____.”

Answer: bad, disappointing, terrible

What’s learned:

- Sentiment analysis
- Sarcasm and implicit meaning
- Opinion and evaluation language

Example 3.6 (Spatial Reasoning). **Masked:** “Iro went to the kitchen to make some tea. Standing next to Iro, Zuko pondered his destiny. Zuko left the _____.”

Answer: kitchen

What’s learned:

- Spatial tracking across sentences
- If Iro is in the kitchen, and Zuko is next to Iro, then Zuko is in the kitchen
- Physical embodiment and location

Example 3.7 (Mathematical Patterns). **Masked:** “The sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, _____”

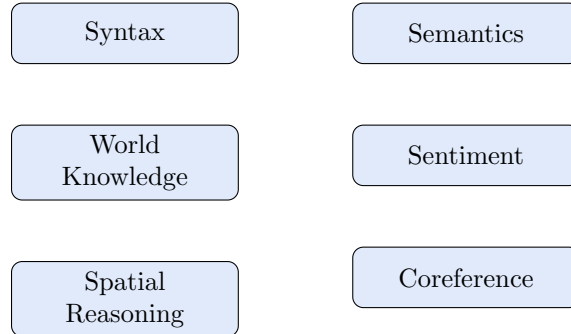
Answer: 34 (Fibonacci sequence)

What’s learned:

- Pattern recognition
- Mathematical reasoning
- Sequential dependencies

Note: This is challenging! The model would need exposure to many examples of mathematical sequences.

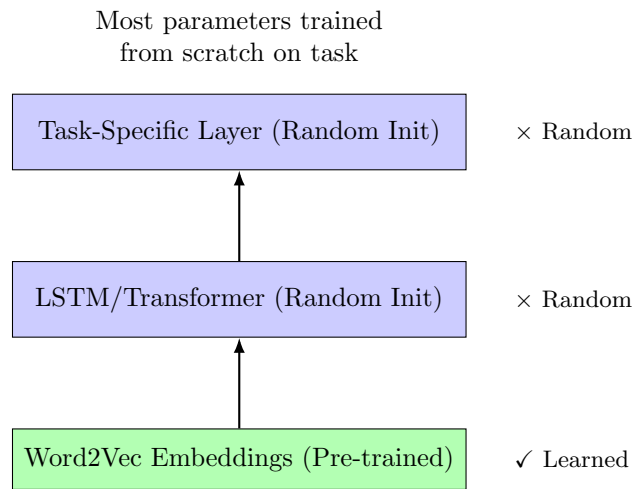
Phenomena Learned from Masked Prediction



4 Pre-training and Fine-tuning Paradigm

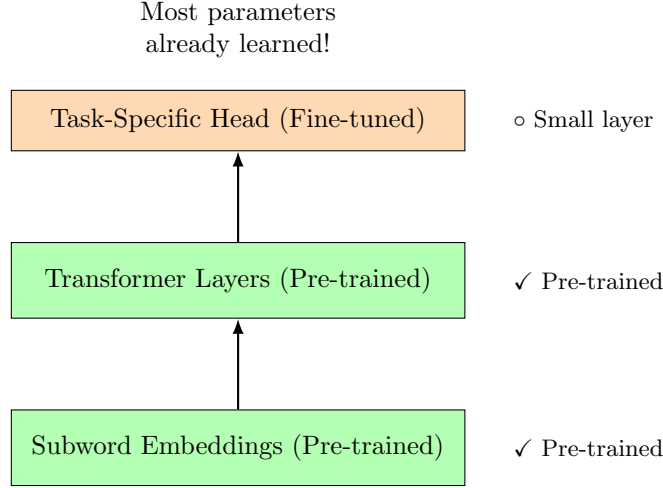
4.1 Traditional Approach: Pre-trained Embeddings Only

Before pre-training era:



4.2 Modern Approach: Full Model Pre-training

Pre-training era:



4.3 Two-Stage Training Process

Task: Language modeling
or masked prediction

Output: $\theta_{\text{pretrained}}$;

[stage, fill=neuralgreen!20, right=3cm of pretrain] (finetune) **Stage 2: Fine-tuning**

Data: Small labeled dataset
(thousands of examples)

Task: Specific NLP task
(sentiment, QA, etc.)

Output: θ_{task} ;

[arrow] (pretrain) – node[above, font=] Initialize (finetune);
[below=0.5cm of pretrain, font=, align=center] **5-10 trillion tokens**
General text from web ;
[below=0.5cm of finetune, font=, align=center] **1K-1M examples**
Task-specific labels ;

4.4 Mathematical Formulation

Pre-training:

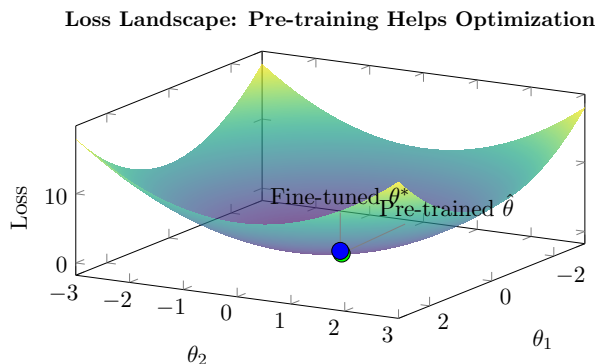
$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}_{\text{pretrain}}(\theta; \mathcal{D}_{\text{unlabeled}}) \quad (1)$$

Fine-tuning:

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{\text{task}}(\theta; \mathcal{D}_{\text{labeled}}) \quad \text{starting from } \theta = \hat{\theta} \quad (2)$$

4.5 Why Does This Work?

1. **Better initialization:** Starting from $\hat{\theta}$ provides much better parameter values than random initialization
2. **Optimization landscape:** The fine-tuning process stays close to the pre-trained parameters, which tend to generalize well



3. **Transfer of knowledge:** Representations learned during pre-training encode general linguistic and world knowledge useful for downstream tasks
4. **Data efficiency:** With good initialization, we need much less labeled data to achieve good performance
5. **Generalization:** Pre-training on diverse text sources helps the model generalize better to new domains

4.6 Comparison with Training from Scratch

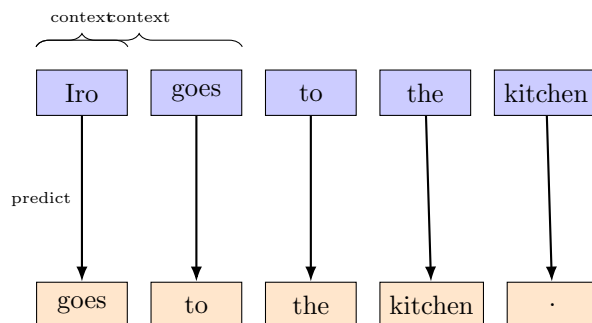
Training from Scratch	Pre-training + Fine-tuning
Requires large labeled dataset	Works with small labeled data
Task-specific knowledge only	General + task-specific knowledge
Poor generalization	Better generalization
Slow convergence	Fast convergence

5 Language Modeling as Pre-training

5.1 Simple Approach: Standard Language Modeling

The most straightforward pre-training objective is standard left-to-right language modeling:

$$\mathcal{L}_{\text{LM}} = - \sum_{t=1}^T \log P(x_t \mid x_1, \dots, x_{t-1}; \theta) \quad (3)$$



Teacher forcing: During training, we always use the gold previous tokens as input, even if the model would have predicted something different.

5.2 After Pre-training: Using the Model

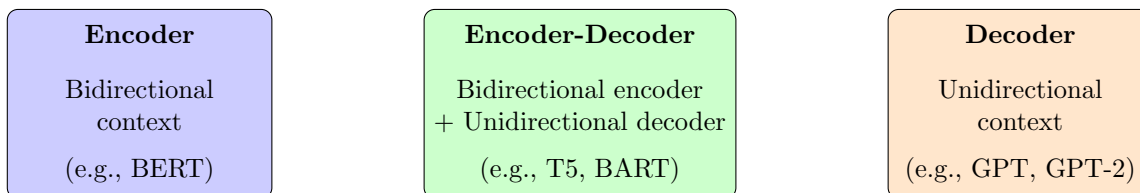
Once pre-trained, the model parameters $\hat{\theta}$ can be used in two ways:

1. **As a language model:** Generate text by sampling from $P(x_t \mid x_{<t}; \hat{\theta})$
2. **As initialization:** Use $\hat{\theta}$ to initialize parameters for a downstream task, then fine-tune

6 Three Types of Model Architectures

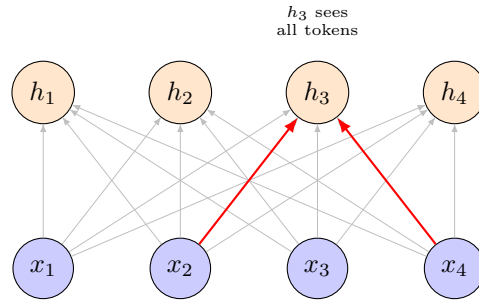
6.1 Architecture Overview

Recall from the Transformers lecture, there are three main architectural patterns:



6.2 Encoder Architecture

Key property: Can attend to entire input in both directions (bidirectional context).



Bidirectional Context:

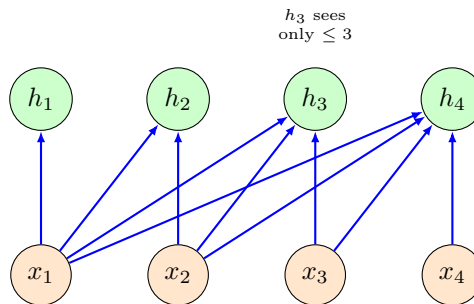
Each position attends to all positions

Good for: Classification, tagging, span extraction (where you need full context).

Bad for: Generation (would trivially copy the next word).

6.3 Decoder Architecture

Key property: Can only attend to previous tokens (unidirectional/causal context).



Unidirectional Context:

Each position attends only to previous positions

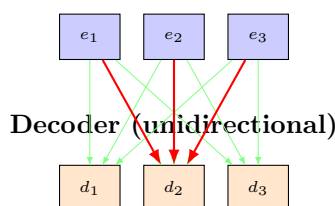
Good for: Generation, language modeling.

Bad for: Tasks requiring full bidirectional context.

6.4 Encoder-Decoder Architecture

Key property: Encoder has bidirectional context, decoder has unidirectional context with cross-attention to encoder.

Encoder (bidirectional)



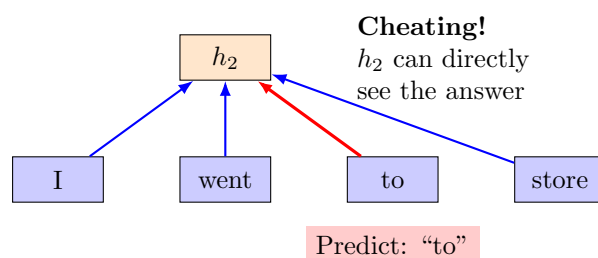
Decoder attends to
all encoder states

Good for: Sequence-to-sequence tasks (translation, summarization).

7 Encoder Pre-training: Masked Language Modeling

7.1 Why Not Standard LM for Encoders?

Problem: If we use left-to-right language modeling with bidirectional attention, the task becomes trivial!

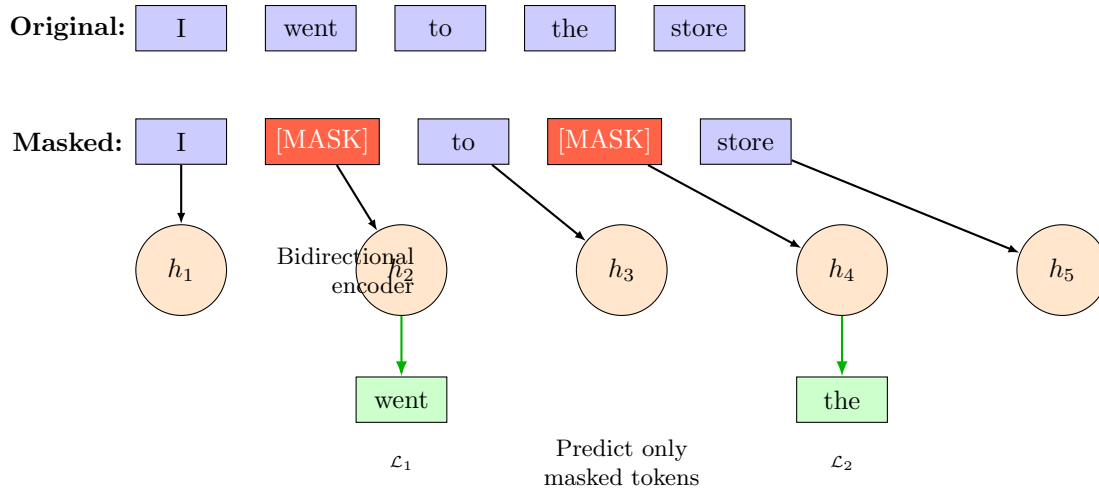


7.2 Solution: Masked Language Modeling (MLM)

Idea: Randomly mask out some tokens and predict them using bidirectional context.

Definition 7.1 (Masked Language Modeling). Given input sequence $X = (x_1, \dots, x_T)$:

1. Create corrupted version $\tilde{X}$ by masking some tokens
2. Model predicts original tokens: $P(X | \tilde{X}; \theta)$
3. Loss computed only on masked positions



Loss function:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \text{masked}} \log P(x_i | \tilde{X}; \theta) \quad (4)$$

8 BERT: Bidirectional Encoder Representations from Transformers

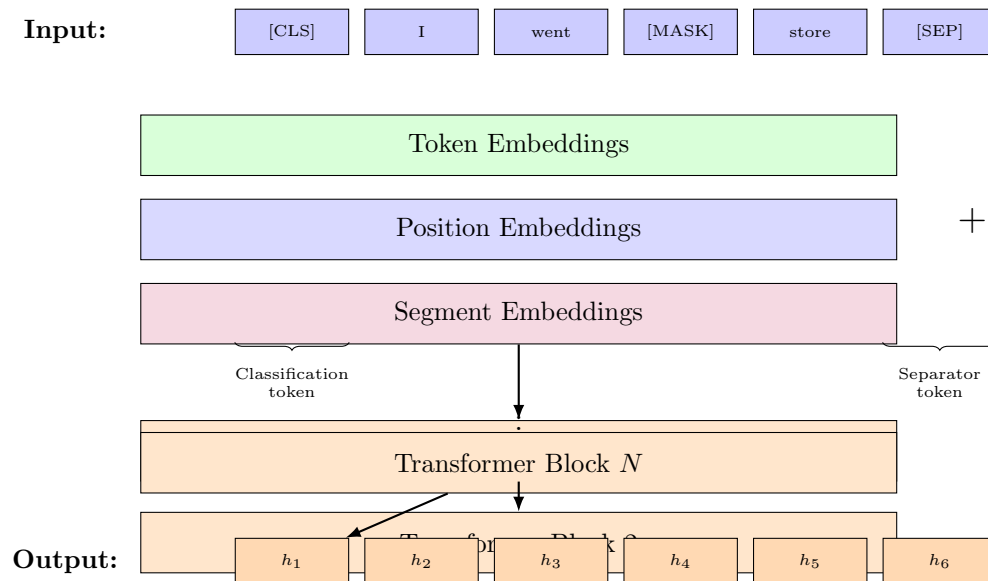
8.1 Overview

BERT (Devlin et al., 2018) was the first model to popularize masked language modeling for pre-training. It caused a paradigm shift in NLP.

Key innovations:

- Masked language modeling objective
- Bidirectional Transformer encoder
- Pre-training on massive text corpus
- Simple fine-tuning for downstream tasks

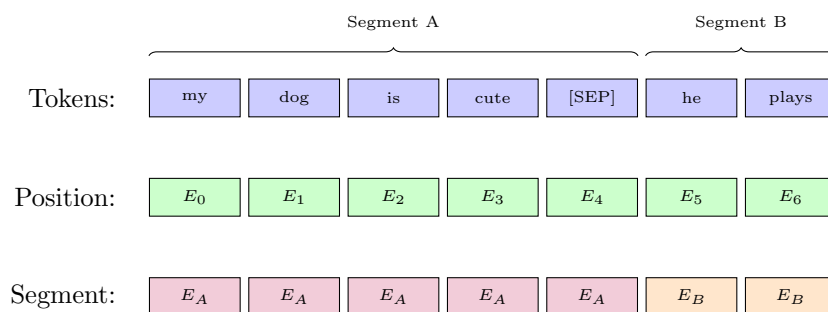
8.2 BERT Architecture



8.3 Three Types of Embeddings

BERT combines three types of embeddings (added element-wise):

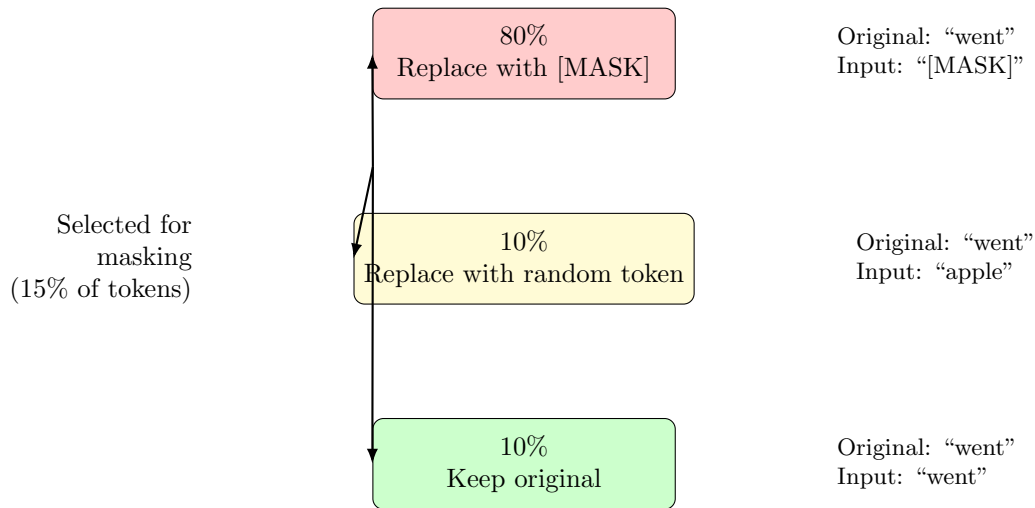
1. **Token Embeddings:** Standard learned embeddings for each subword token
2. **Position Embeddings:** Learned embeddings encoding position in sequence
 - Unlike Transformers lecture's sinusoidal encoding
 - BERT uses learned position embeddings
 - Maximum sequence length: 512 tokens
3. **Segment Embeddings:** Binary embedding indicating which segment (A or B)
 - Used for next sentence prediction task
 - Two possible values: E_A or E_B



8.4 Masking Strategy

BERT doesn't simply replace 100% of masked tokens with [MASK]. Instead, for each token selected for masking (15% of all tokens):

- **80%:** Replace with [MASK]
- **10%:** Replace with random token
- **10%:** Keep original token



Rationale:

- If we only use [MASK], at fine-tuning time there are no [MASK] tokens!
- Random replacement: Model must detect if word is wrong
- Keep original: Model must be ready to represent any token
- Forces model to maintain good representations for all tokens

8.5 Next Sentence Prediction (NSP)

BERT originally included a secondary task: predicting if sentence B follows sentence A.

Positive Example (50%)

Segment A: "Iro went to the kitchen."

Segment B: "He made some tea."

Label: IsNext

Negative Example (50%)

Segment A: "Iro went to the kitchen."

Segment B: "The sky is blue."

Label: NotNext

Note: Later research (RoBERTa) showed NSP is not necessary and can hurt performance. Modern models often skip this task.

Property	BERT-Base	BERT-Large
Layers (N)	12	24
Hidden size (d)	768	1024
Attention heads (h)	12	16
Parameters	110M	340M
Training data	BooksCorpus + Wikipedia ≈ 3.3 billion words	
Training time	4 days on 64 TPUs	

Table 1: BERT model configurations

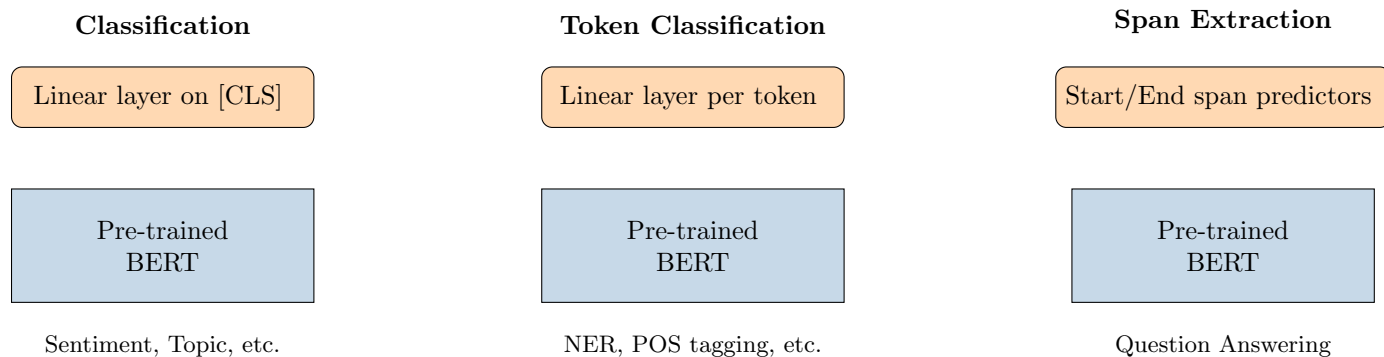
8.6 BERT Training Details

Pre-training was expensive, but fine-tuning is practical:

- Fine-tuning can be done on a single GPU
- Takes minutes to hours depending on dataset size
- Much more accessible than pre-training

8.7 Fine-tuning BERT

The beauty of BERT is how easily it adapts to downstream tasks:



General recipe:

1. Take pre-trained BERT
2. Add small task-specific layer on top
3. Fine-tune entire model on task data
4. Achieve state-of-the-art performance!

9 BERT Evaluation: GLUE Benchmark

9.1 The GLUE Benchmark

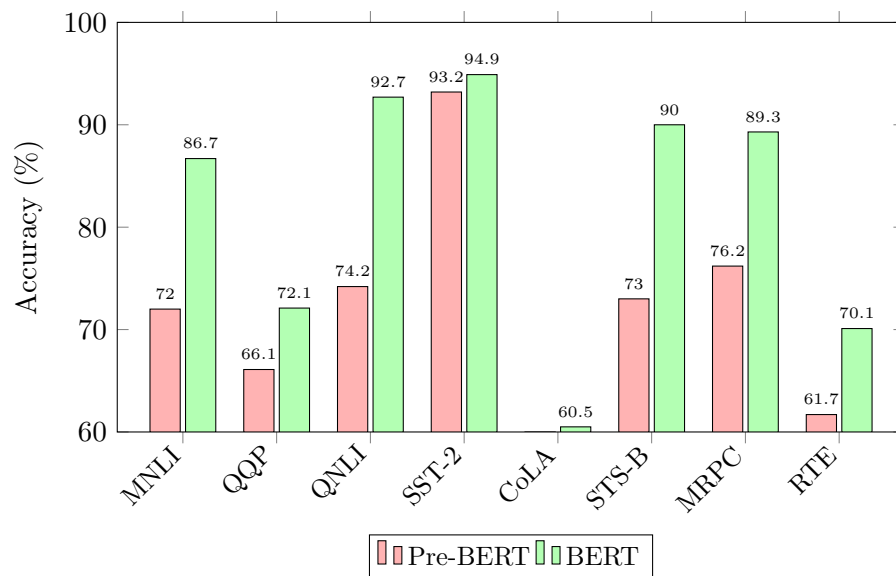
GLUE (General Language Understanding Evaluation) is a collection of 9 diverse NLP tasks designed to test language understanding.

Task	Description
MNLI	Multi-genre natural language inference
QQP	Quora question pairs (paraphrase detection)
QNLI	Question answering NLI
SST-2	Sentiment analysis (Stanford Sentiment Treebank)
CoLA	Linguistic acceptability judgments
STS-B	Semantic textual similarity
MRPC	Microsoft paraphrase corpus
RTE	Recognizing textual entailment
WNLI	Winograd schema challenge

Table 2: GLUE benchmark tasks

9.2 BERT's Impact

BERT dramatically improved performance across all GLUE tasks:



Key observations:

- Massive improvements on most tasks (5-25% absolute gain)
- Especially strong on tasks requiring understanding of relationships between sentences
- Simple fine-tuning beats carefully designed task-specific architectures

10 Limitations and Considerations

10.1 When to Use BERT

BERT is great for:

- Classification tasks (sentiment, topic, toxicity)
- Token-level tasks (NER, POS tagging)
- Span extraction (question answering)

- Tasks requiring full bidirectional context

BERT is NOT ideal for:

- Text generation (use decoder models like GPT)
- Very long documents (512 token limit)
- Real-time applications requiring low latency

10.2 The [MASK] Token Problem

Problem 10.1 (Train-Test Mismatch). During pre-training, BERT sees [MASK] tokens. During fine-tuning and inference, there are no [MASK] tokens! This creates a distribution mismatch.

Mitigation: The 80-10-10 masking strategy helps, but doesn't fully solve the problem.

10.3 Computational Costs

Pre-training: Very expensive

- Days/weeks on many GPUs/TPUs
- Millions of dollars in compute

Fine-tuning: Practical

- Hours on single GPU
- Hundreds of dollars

Inference: Moderate cost

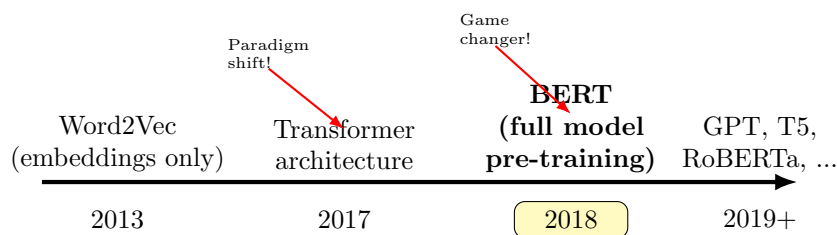
- Slower than smaller models
- Can be expensive at scale

11 Summary and Key Takeaways

11.1 Main Concepts

1. **Subword Tokenization:** Solve OOV problem by splitting words into frequent subword units (BPE)
2. **Pre-training Paradigm:** Train large models on huge unlabeled corpora, then fine-tune on small labeled datasets
3. **What's Learned:** Masked prediction teaches models about syntax, semantics, world knowledge, and reasoning
4. **BERT:** Bidirectional Transformer encoder trained with masked language modeling
5. **Transfer Learning:** Pre-training provides excellent initialization for downstream tasks

11.2 The Pre-training Revolution



11.3 Looking Ahead

In the next lecture (Pre-training Part 2), we'll cover:

- Decoder-only models (GPT family)
- Encoder-decoder models (T5, BART)
- Scaling laws and emergent abilities
- Modern pre-training techniques
- Prompt-based learning

12 Further Reading

- **BPE:** Sennrich et al. (2016). “Neural Machine Translation of Rare Words with Subword Units”
- **BERT:** Devlin et al. (2018). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”
- **RoBERTa:** Liu et al. (2019). “RoBERTa: A Robustly Optimized BERT Pretraining Approach”
- **GLUE Benchmark:** Wang et al. (2018). “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding”